R1.02 — Développement d'interfaces web : L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Architecture Web, Protocole HTTP et Modèle Client-Serveur

Synthèse exécutive : Ce cours pose les fondements indispensables du fonctionnement du Web en détaillant l'architecture client-serveur et le protocole HTTP (HyperText Transfer Protocol). L'étudiant y découvre comment un navigateur formule des requêtes standardisées, comment le serveur y répond de manière asynchrone et sans état (stateless), et comment analyser les codes de statut HTTP pour diagnostiquer et déboguer des applications web.

L'Architecture Client-Serveur et le Principe de Communication

L'architecture client-serveur est le modèle fondamental du Web. Contrairement au pair-à-pair (P2P), les rôles sont stricts. Le client (Firefox, Chrome, Safari) envoie une requête HTTP à travers le réseau. Le serveur web (Apache, Nginx, IIS) écoute passivement, traite la requête, accède éventuellement à des ressources ou à une base de données, puis renvoie une réponse HTTP contenant la ressource (HTML, CSS, image, JSON) ou une erreur.

- Client (Navigateur) : Logiciel actif qui initie les requêtes.
- Serveur Web : Processus logiciel passif en écoute sur un port spécifique (80 pour HTTP, 443 pour HTTPS).
- Communication Asynchrone : Échange initié par le client, traité par le serveur, puis clôturé par la réponse.

Cas concret enseignant : *Le professeur explique que lorsqu'un étudiant se connecte sur le site de l'IUT de Bayonne, son navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer le fichier HTML de base, puis chaque image, feuille de style CSS et script JavaScript associé.*

Le Protocole HTTP et sa Nature Stateless (Sans État)

Le protocole HTTP est 'stateless' (sans état). Le serveur ne conserve aucune trace des requêtes précédentes d'un même client. Cela simplifie grandement l'architecture et améliore la scalabilité des serveurs, mais pose un problème pour les applications nécessitant un suivi (connexion utilisateur, panier d'achat). Des mécanismes comme les cookies et les sessions comblent cette lacune.

- Protocole Stateless : Chaque échange requête-réponse est totalement indépendant et sans mémoire des interactions passées.
- Mécanismes de Session : Utilisation de cookies (côté client) et de sessions (côté serveur) pour contourner l'absence d'état natif de HTTP.

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un inconnu. Pour qu'il se souvienne de vous, vous devez présenter un badge (le cookie de session) à chaque échange.*

Structure des Messages HTTP : Requêtes, Réponses et Méthodes

Une requête HTTP est un message textuel structuré comportant une ligne descriptive, des en-têtes, une ligne vide et un corps optionnel. La réponse adopte une structure similaire mais commence par une ligne de statut. GET et POST sont les méthodes principales : GET récupère une ressource sans modifier l'état du serveur, tandis que POST soumet des données pour modifier l'état.

- Ligne de requête : Contient la méthode (GET, POST), l'URI et la version du protocole.
- En-têtes (Headers) : Métadonnées clés-valeurs décrivant le client, le serveur ou le contenu.
- Corps (Body) : Zone optionnelle contenant les données brutes (formulaires, JSON).
- GET vs POST : GET est pour la consultation (paramètres visibles dans l'URL), POST est pour la modification (paramètres dans le corps).

Cas concret enseignant : *Le professeur insiste sur le fait que le protocole HTTP est purement textuel et lisible en clair sans HTTPS. Utiliser GET pour envoyer un mot de passe constitue une faille de sécurité majeure visible dans l'historique ou par shoulder surfing.*

La Classification des Codes de Statut HTTP

Chaque réponse HTTP commence par un code numérique à trois chiffres permettant au client de comprendre le résultat. Il faut impérativement distinguer les erreurs 4xx (faute du client, ex: mauvaise URL) des erreurs 5xx (faute du serveur, ex: bug de script ou base de données inaccessible).

- Codes 2xx (Succès) : Requête reçue et traitée avec succès (ex: 200 OK).
- Codes 3xx (Redirection) : Actions supplémentaires requises (ex: 301 Moved Permanently).
- Codes 4xx (Erreur Client) : Anomalie dans la requête du client (ex: 404 Not Found, 403 Forbidden).
- Codes 5xx (Erreur Serveur) : Échec interne du serveur (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, un point-virgule manquant dans son script PHP plantait la base de données à chaque soumission de formulaire, renvoyant une erreur 500. Les utilisateurs croyaient le site piraté.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : GET transmet les paramètres dans l'URL (visible, limitée en taille, mise en cache possible), tandis que POST transmet les données de manière invisible dans le corps de la requête. Pour un mot de passe, POST est obligatoire pour des raisons évidentes de sécurité.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante et identifier la méthode, la ressource demandée, l'hôte cible, et le comportement en cas d'absence de la ressource : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyse de la ligne de requête : 'GET /images/logo.png HTTP/1.1'. Le verbe est 'GET' (récupération de ressource), l'URI est '/images/logo.png'. 2. Identification de l'hôte : L'en-tête 'Host: iutbayonne.univ-pau.fr' indique le domaine cible. 3. Analyse de l'agent : 'User-Agent: Mozilla/5.0' identifie le navigateur client. 4. Scénario d'absence : Si 'logo.png' est introuvable sur le serveur, celui-ci doit retourner une erreur de la famille 4xx.

Résultat : Méthode : GET | Ressource : /images/logo.png | Hôte cible : iutbayonne.univ-pau.fr | Code en cas d'absence : 404 Not Found (Erreur Client)

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?

R : Un protocole où chaque échange requête-réponse est indépendant et où le serveur ne conserve aucune mémoire des interactions passées.

Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?

R : Les codes 4xx indiquent une erreur imputable au client (ex: URL incorrecte), tandis que les codes 5xx indiquent une défaillance interne du serveur.

Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?

R : Parce que GET place les paramètres directement dans l'URL, les rendant visibles dans l'historique du navigateur, les logs et lors d'une observation directe de l'écran.